MORSE-BASED MODULAR HOMOLOGY FOR EVOLVING SIMPLICIAL COMPLEXES: SUPPLEMENTARY MATERIAL

A PREPRINT

Anqiao Ouyang
Mount Boucherie Secondary School
West Kelowna, British Columbia, Canada

Department of Aerospace Engineering
San Diego State University
San Diego, California, USA
aouyang@sdsu.edu

October 12, 2026

S1 Setup details

This supplement accompanies the main paper. Sections, tables and figures numbered with an S are in this document; all other theorems, sections, tables and figures are those of the main paper. The experiments are organised in the configuration files by the identifiers E0–E8 of Table S1; this supplement uses them to point to the data.

S1.1 Implementation and platform

Engine. All methods except gudhi run in one C++17 program and share its complex store, which maps the sorted vertex tuple of a simplex to its facets and cofaces and validates every atomic edit. Order keys are dense 32-bit integers; a (re)build assigns $0, \dots, N-1$ in the chosen order, and every later insertion takes the next unused key (Section 3). A chain is a strictly increasing array of keys, so its low is the last entry and a $\mathbb{Z}_2$ addition is a linear merge; a working column that receives many short chains switches to a max-heap with lazy cancellation. Deletions therefore pay the sorted-array cost of Proposition 3, and insertions its ordered-set cost up to a logarithmic factor. The footprint index keeps the exact count $|\operatorname{occ}(\sigma)|$ and a lazily filtered list of slots. In Case A the engine chooses the cycle V_z with the fewest cells (ties: the smallest slot identifier). Only the methods that may compute a Morse order after K_0 keep an index of the cells by vertex tuple; among the DRB methods that is mmhm. The collapse of Definition 4.1 uses a first-in-first-out queue initialised in (dimension, vertex tuple) order, so the matching depends only on the complex. Debug builds check (I1)–(I3), the pivot table and the footprint index after every operation.

Platform. All runs use one cloud virtual machine with an Intel Xeon processor at 2.10 GHz (4 cores, one thread per core; 48 KiB L1 data and 2 MiB L2 cache per core, 260 MiB shared L3 cache) and 15 GiB of memory, running Ubuntu 24.04.5 LTS with Linux 6.18. The engine is compiled by g++ 13.3.0 (CMake 3.28.3) with `-O2 -DNDEBUG` and no machine-specific tuning. The GUDHI baseline and oracle run under Python 3.13.16 with GUDHI 3.13.0 and numpy 2.5.3. Every method runs single-threaded.

S1.2 Further notes on the methods

Static baselines. full-id, full-sorted and full-morse recompute all ranks after every update by the static reduction with clearing (Section 3), without storing V: full-id in insertion order, full-sorted in the canonical order, full-morse in the Morse order of a collapse matching computed anew after every update. By Corollaries 2.1 and 2.2 the column additions

of full-morse all occur in critical columns and, after the gradient-flow steps, are those of the reduction of the Morse complex, but it pays for a collapse after every update. The insertion order of full-id is the order in which the cells now present were inserted, so it depends on the history of edits: refilled cells receive late keys (Section S1.4 reports the consequence). The canonical order of full-sorted depends only on the current complex, so it cannot drift with the history; it is our history-independent reference. All three share the store and the reduction code with the dynamic methods, so differences reflect mostly the algorithms; the remaining implementation differences are this history dependence and the index of cells by vertex tuple that full-sorted and mmhm maintain (E4). gudhi replays the same edits on a GUDHI simplex tree [1] through its Python interface and, after every update, calls `compute_persistence` with the arguments `homology_coeff_field=2` and `persistence_dim_max=True`, and then `betti_numbers`; without the second argument GUDHI omits the top dimension. GUDHI computes persistence pairs, a richer output, and is not designed for this task; it serves as a reference point for a widely used library and as an oracle.

Dynamic methods. mmhm is the method of Sections 3 and 4: a dynamic reduced basis built in the Morse order and rebuilt in a fresh Morse order under the rebuild policy of Section 3, with $m = \infty$ and $\alpha = 1$ unless stated otherwise (E7 varies both). Three ablations separate its ingredients. drb-id uses insertion keys and never rebuilds. mmhm-norb uses the Morse order of K_0 and never rebuilds. drb-id-rb rebuilds under the same policy and parameters as mmhm, but its rebuild reruns the static reduction with clearing in the current insertion order (the keys in use, renumbered) instead of computing a new matching. So mmhm against drb-id-rb isolates the order used at a rebuild, and mmhm against mmhm-norb isolates rebuilding. drb-rand builds the DRB in a uniformly random order of the cells of each dimension, drawn from the seed, and never rebuilds; against drb-id and mmhm-norb it tests whether a structured order matters at all (E8).

Surface counters. euler maintains χ and the numbers of edges with one, two or more cofaces in $O(1)$ per atomic edit and returns the Betti numbers of Remark 5.1. At the end of every update it checks that the complex has a triangle, no tetrahedron, and no edge outside one or two triangles, and aborts otherwise; connectivity and the absence of pinched vertices are assumed, not checked.

S1.3 Workload generators

Five generators produce the update sequences. Each is deterministic given its parameters and seed, emits only valid atomic edits (checked again by an independent validator), lists deletions top-down and insertions bottom-up, and records its parameters, the sizes of all states and the number of edits per update. For W1, W2, W3 and W5 the Betti numbers of every state are known in closed form and are written next to the sequence. Distances are graph distances in the base mesh.

W1: punctured surfaces. The base surface is the geodesic icosphere of frequency $f = 2^L$, with $10f^2 + 2$ vertices, $30f^2$ edges and $20f^2$ triangles, or the $n \times m$ periodic grid torus with every square split along one diagonal, with nm vertices, $3nm$ edges and $2nm$ triangles. For a vertex v, the patch $P(v, k)$ is the set of triangles incident to a vertex at distance at most $k - 1$ from v; we use $k = 2$. When v and its neighbours have degree six, $P(v, 2)$ has 24 triangles and opening it takes 61 atomic edits; no vertex of our surfaces has a larger degree, so no update has more edits. A patch around a random vertex is used only if its closure is a disk and its vertices are at distance more than one from the vertices of every existing hole. Opening a hole deletes the triangles of the patch, then its edges, then the vertices left without cofaces; closing a hole is the exact reverse of its opening. With h holes, the generator opens a hole if $h = 0$; otherwise it opens one with probability 0.75, 0.5 or 0.25 when h is below, at or above the target 3, and closes one otherwise or when no valid patch is found. The Betti numbers are $(1, \max(h - 1, 0), [h = 0], 0)$ for the sphere and $(1, 2 + \max(h - 1, 0), [h = 0], 0)$ for the torus, where $[h = 0]$ is 1 if $h = 0$ and 0 otherwise. W1 is the setting of Remark 5.1, which E5 examines. It also contains the phenomenon of Proposition 4: when no hole is open, the surface is a connected closed surface, so the deletion that opens the first hole costs every DRB method $\Omega(n_2)$ operations by Proposition 4(a), and the last triangle inserted when the last hole closes costs $\Omega(n_2)$ by Proposition 4(b).

W2: refinement and coarsening in a 3-ball. The base complex is the Freudenthal (Kuhn) triangulation of a grid of $N \times N \times N$ cubes, each split into six tetrahedra; it has $(N + 1)^3$ vertices and about $26N^3$ simplices. A refinement applies s stellar $1 \to 4$ subdivisions to s tetrahedra collected breadth-first over face adjacency from a uniformly random tetrahedron; tetrahedra created by earlier subdivisions may be subdivided again. A coarsening undoes s leaf subdivisions, those whose four tetrahedra are all still present: a uniformly random one and the $s - 1$ nearest to it, measured by the Euclidean distance between the inserted vertices. A subdivision and its inverse each consist of 16 atomic edits, so $s_t = 16s$. Because coarsening can only undo subdivisions, K_0 already contains $8s$ live subdivisions, and with x live subdivisions the generator refines with probability $\min\big(0.95, \max(0.05,\ 0.5 - 0.5(x - 8s)/(8s))\big)$, and always when fewer than s leaf subdivisions exist. Since $|K| = |K_{\text{base}}| + 14x$, the size performs a random walk

Table S1: Experiments. Parameters not listed are the generator defaults of Section S1.3. "as E1": full-id, full-sorted, full-morse, gudhi, drb-id, drb-id-rb, mmhm, mmhm-norb; "DRB methods": the last four. gudhi processes all updates on instances with at most 300,000 simplices and the first $T' = 200$ updates on larger ones.

	Workload	Sweep	Methods
E0	samples; all runs of E1–E8	—	all exact methods
E1	W2, $s = 8$	$N \in \{7, 10, 15, 22, 33\}$	as E1
E2	W2, $N = 22$	$s \in \{1, 2, 4, 8, 16, 32, 64\}$	DRB methods, full-id, full-morse
E3	W3, $k = 2$	$N \in \{14, 20, 27\}$	as E1
E4	W4	square, $D = 2$: $n \in \{1000, 4000, 16000\}$; square and cube, $D = 3$: $n \in \{1000, 4000\}$	as E1
E5	W1	sphere $L \in \{4, 5, 6, 7\}$; torus $n = m \in \{64, 128, 256, 500\}$	as E1, and euler
E6	W5a, W5b	(a) $L = 10^3, 3 \cdot 10^3, 10^4, \ldots, 10^6$; (b) levels $L = 3, \ldots, 8$	DRB methods, full-id, gudhi; euler on (b)
E7	W2 $N = 22$, W3 $N = 20$, W4 square $n = 16000$, $D = 2$	$\alpha \in \{0.25, 0.5, 1, 2, 4, \infty\} \times m \in \{\infty, 256, 64\}$	mmhm, drb-id-rb
E8	W2 $N = 22$, W3 $N = 20$, W4 as in E7, W1 torus 256×256; for drb-rand one size per family (Table S14)	runs of E1, E3, E4, E5	DRB methods; drb-rand

around its initial value. The Betti numbers are $(1, 0, 0, 0)$ after every update, but not within an update: by Theorem 1(d) every atomic edit changes one Betti number by one.

W3: cavities in a 3-ball. The base complex is that of W2. A centre v is drawn uniformly among the interior vertices, and the ball $B(v, k)$ is the set of tetrahedra incident to a vertex at distance at most $k - 1$ from v; we use $k = 2$. Carving deletes the tetrahedra of B and then, top-down, every face left without cofaces. A ball is accepted only if it is more than one ring away from every carved ball, its boundary is a 2-sphere, and it meets the boundary of the cube in no triangle (a cavity: β_2 grows by one) or in one disk (a dent: no change); other contacts, which could bore a tunnel, are rejected. Refilling is the exact reverse of a carving. The number of carved balls follows the same walk as the number of holes in W1, with target 4. The Betti numbers are $(1, 0, c, 0)$, where c is the number of cavities.

W4: flag complexes of moving points. n points are drawn uniformly in the unit square or the unit cube ("2D" and "3D" in the tables and figures). A radius ε is calibrated on the initial points so that the ε-neighbourhood graph has exactly $5n$ edges (mean degree 10), and K is the flag complex of this graph truncated at dimension $D \in \{2, 3\}$. An update moves one uniformly random point by adding to each coordinate an independent offset drawn uniformly from $[-\varepsilon/4, \varepsilon/4]$, reflected at the border of the domain, and keeps its vertex identifier: it deletes the closed star of the vertex top-down and inserts the vertex and its new star bottom-up. The complexes are not manifolds and all Betti numbers vary; they are not known in closed form and are checked against the oracles only.

Caveat: truncation. In a flag complex truncated at dimension D, the vertices of a clique of size $D + 2$ span a $(D + 1)$-simplex that is not in K, so its boundary is a D-cycle that is never filled. Hence β_D is large: at INIT, β_D/n lies between 2.8 and 6.2 in our W4 instances (Table S13). This is genuine linear algebra on a non-manifold complex, but β_D is dominated by the truncation, not by the shape of the point cloud. We use W4 as an algebraic stress test and draw no conclusion about the topology of point clouds from it.

W5: non-locality witnesses. (a) The cycle graph C_L, whose updates toggle the edge $\{0, 1\}$: odd-numbered updates delete it and even-numbered ones insert it again. (b) The icosphere of frequency 2^L, whose updates toggle one triangle chosen from the seed in the same way. By Proposition 4, every toggle costs every DRB method $\Omega(L)$, respectively $\Omega(n_2)$, operations; once the states alternate, a toggle costs $O(L + \mathrm{nnz}(R) + \mathrm{nnz}(V))$, respectively $O(n_2 + \mathrm{nnz}(R) + \mathrm{nnz}(V))$. euler needs $O(1)$ per toggle on (b) and does not apply to (a), which is a graph and not a surface.

Sweeps. Table S1 lists the workloads, parameters and methods of each experiment. Every timed configuration ran with 3 seeds.

S1.4 Timing protocol and drift of the static baseline

The input sequence is parsed completely before timing starts. For the engine methods, the time of update t is the wall-clock time (`std::chrono::steady_clock`) of processing its block of atomic edits: the store updates, the work of the method for each edit, the recomputation of the static baselines, and any rebuild, which is charged to the update that triggers it. Verification and output are excluded. Building the structure for K_0 is timed separately and reported as build time. For gudhi, the time of an update covers the simplex-tree edits, `compute_persistence` and `betti_numbers`, again without parsing.

All runs are single-threaded processes. Each configuration ran with 3 seeds, which change the generated workload; W5a is determined by L alone, so its three runs are repetitions of one instance. Every run processed $T = 2{,}000$ updates, except gudhi on instances with more than 300,000 simplices, which processed the first $T' = 200$ (hollow markers in the figures, $'$ in the tables). For each instance and method we pool the per-update latencies of the three runs and report their median, interquartile range and 95th and 99th percentiles (p95, p99), and the *amortised latency*: the total processing time of all updates, rebuilds included, divided by their number (the initial build is not included; we report it separately). A *speedup* is a ratio of amortised latencies over the same updates; for gudhi on a prefix, both methods are taken over that prefix. Slopes are least-squares fits of $\log$ latency against $\log$ size. All 1,344 configured runs finished (0 were not run); they took 6.5 hours of single-threaded processing in total.

Drift of the static baseline. Because the insertion order of full-id depends on the history, its cost is not stationary. Its median latency over the last 200 updates exceeds that over the first 200 by more than 25 % on 10 instances: every W3 instance (up to a factor 2.86), every W4 instance with $n < 16000$, and the 64×64 torus (Table S7). full-sorted, whose order cannot drift, drifts by at most 1.09 on W1 and W2, by 1.01 to 1.23 on W4 (full-id: 1.15 to 1.64), and by 1.44 to 1.84 on W3, where part of the drift therefore comes from the changing complex rather than from the order. The DRB methods drift as well; on W3 mmhm drifts less than full-id. Where the drift matters we give speedups over matched windows of the first and the last 200 updates and against full-sorted.

Counters. Besides time, every run records per update the number of column additions, the sum of the footprints $|\operatorname{occ}(\sigma)|$ over the deleted simplices, and $\operatorname{nnz}(R) + \operatorname{nnz}(V)$; in an update with a rebuild, the additions of the rebuild and of the edits are summed. These counts are proxies for the cost terms of Proposition 3: they count terms, not the sizes of the chains involved (Section 5.1), so they need not move with time. They do not depend on the machine.

S2 Noise and an outlier run

Noise. Five identical runs of mmhm and full-id on W2 with $N = 22$ gave coefficients of variation (CV) of the amortised latency of 8.4 and 4.5 % and of the p99 of 12.0 and 14.0 %. On the W1 torus 256×256, W3 with $N = 20$ and W5a with $L = 10^5$ (five runs each of mmhm, drb-id and full-id) the amortised CV was 0.6 to 6.7 % and the p99 CV 1.1 to 10.5 %; on microsecond-scale updates (W5a, $L = 1000$) the CV reaches 28.9 % (amortised) and 39.6 % (p99). At most two timed runs executed at once. Concurrent load changed the median within the run-to-run spread at $N = 22$, but at 955,059 simplices three busy processes raised it by $+14.2$ % (full-id) and $+2.5$ % (mmhm); untimed verification jobs overlapped 46 timed runs early in the sweep (`experiments/ENVIRONMENT.md`). We call a difference between two methods *within noise* when their per-seed ranges overlap (`experiments/results/summary_pairs.csv`). Each range has three points, and on W1–W4 the seeds are different instances, so this is a weak criterion, not a test.

The outlier run is described in Section S4.1; Table S2 lists the repeated runs.

S3 Detailed validation

The reported Betti numbers are compared with up to three references: (1) the engine's full-id, a static recomputation that shares the parser and the complex store with the dynamic methods but none of their reduction code, and is therefore *engine-internal*; (2) the closed-form Betti numbers of W1, W2, W3 and W5; and (3) GUDHI, an independent library in another language, which replays the same sequence. The two external references are (2) and (3). A pure-Python implementation of the dynamic reduced basis and of the Morse order, written from the specification and sharing no code with the engine, ran only on the 8 committed samples.

Every exact run records the Betti numbers it reports after every update. full-id processes all T updates on every instance, and every other exact run is compared state by state with the full-id output of the same instance. The runs are also compared with the closed-form Betti numbers where these are known (W1, W2, W3, W5), and with the GUDHI oracle on all updates of instances with at most 300,000 simplices and on the first T' updates of larger ones. In addition, every

exact engine run compares itself with an internal recomputation after K_0, after every 50th update and after the last one, outside the timed region. On the 8 committed samples, all exact engine configurations (including aggressive rebuild settings), the reference implementation, GUDHI and the closed-form values are cross-checked on every state. A mismatch of an exact method stops the whole pipeline. Table 3 gives the resulting coverage.

Below the level of outputs, the engine's randomised tests check (I1)–(I3) and compare every exact method with a brute-force rank computation on random edit sequences, and the reference implementation checks (I1)–(I3) after every atomic edit, Lemma 2 and Corollaries 2.1 and 2.2 on random complexes and Proposition 4 on the W5 families.

S3.1 Results (E0)

Every exact method reported correct Betti numbers in every checked state, and every state was checked against at least one reference.

The 1,290 runs of exact methods reported 2,532,690 Betti vectors (Table S3). Every one was compared with at least one reference, 94.8 % with at least two, 98.3 % with at least one external reference (closed form or GUDHI) and 52.4 % with all three (Table 3); no comparison found a mismatch (0). The 42,021 vectors checked only against the engine-internal full-id are those of the gudhi runs on W4, which has no closed form. In addition, the exact engine runs made 47,724 internal verifications against a recomputation, and on the 8 committed samples the Python reference checked 2,514 vectors of 3 methods (0 mismatches).

S4 Complete performance results

S4.1 W2: dependence on the size of the complex (E1)

Only weakly for the dynamic reduced basis, and linearly for static recomputation. On W2 with $s = 8$, the median per-update latency of the four DRB methods grows with $|K_0|$ with slope 0.24 to 0.27 over five sizes, against 1.04 for full-id, 1.06 for gudhi and 1.14 for full-morse (Figure S1(a)).

At the largest size, $|K_0| = 955{,}059$, the four DRB variants are 90.2 to 91.5 times faster than full-id, and their differences are within noise: no rebuild fired, and the insertion order is as fast as the Morse order. On W2 the gain is that of the dynamic reduced basis, not of the Morse order or of rebuilds. mmhm takes 165 µs per update amortised against 14,900 µs for full-id, and its p99 is 333 against 21,600 µs (Tables S4 and S12). Over the same first T' updates, mmhm is 948 times faster than gudhi. The DRB pays more at the start of a run: over the first 200 updates mmhm is 46.0 times faster than full-id, over the last 200 103 times, although its median hardly changes (138 and 143 µs); a few early updates are expensive. full-sorted, which does not drift on W2 and is within noise of full-id at this size, gives the same picture: mmhm is 88.6 times faster over the whole run, 44.2 and 100 times over the two windows. Building the structure for K_0, including the insertion into the store, took 719 ms for mmhm and 667 ms for full-id, and $\mathrm{nnz}(R) + \mathrm{nnz}(V)$, a proxy for the memory of the DRB, reached at most 3.80 entries per simplex.

One run is an outlier: at $N = 22$, one of the three mmhm-norb runs was slow in its first 400 updates only (median 581 µs, against 118 and 125 µs for the other two). A rerun produced identical Betti numbers, additions, nnz and footprints and normal timings (amortised 114 against 252 µs); the tables keep the original run.

Static recomputation in the Morse order. full-morse is slower than full-id and full-sorted on every instance on which they ran, than full-id by factors from 1.09 (W1 sphere, $L = 4$) to 4.22 (W2, $N = 33$; Table S10). There is little for the collapse to save: on W2 the insertion order needs only 192 additions per recomputation, and on the largest W4 instance full-morse performs more additions than full-id (429,000 against 398,000). For one-shot static recomputation the collapse does not pay for itself on our workloads.

S4.2 W2: dependence on the edit size (E2)

Linearly for the DRB methods. On W2 with $N = 22$ ($|K_0|$ between 285,805 and 292,861), the median per-update latency of the DRB methods grows with the number s of stellar moves with slope 0.88 to 0.90, whereas full-id and full-morse barely depend on s (slopes 0.06 and 0.07; Figure S2(a)). The amortised cost per atomic edit of the DRB methods therefore stays in a narrow band, while that of recomputation falls roughly as $1/s$ (Figure S2(b)). There is no crossover in the measured range: at $s = 64$, that is 1024 atomic edits per update, the DRB methods are still 4.80 to 5.85 times faster than full-id.

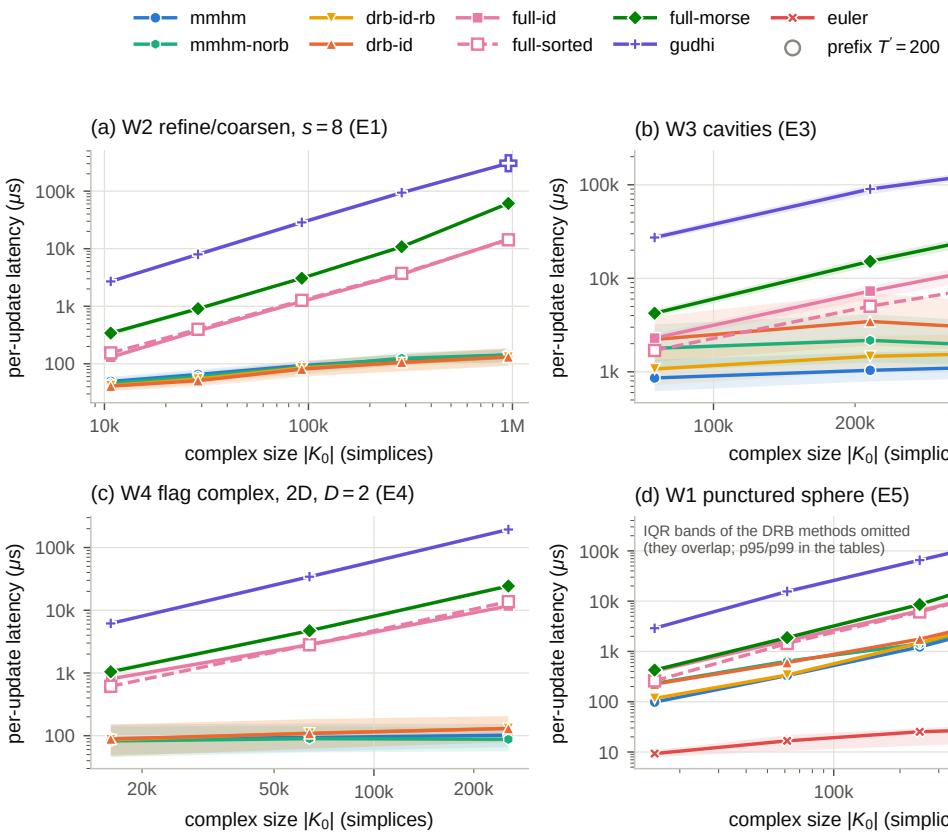


Figure S1: Median per-update latency against $|K_0|$ at a fixed edit size (log–log; band: interquartile range; three runs pooled). (a) W2 with $s = 8$ (E1); (b) W3 (E3); (c) W4 in the square with $D = 2$ (E4); (d) W1 punctured sphere (E5), without the bands of the DRB methods, which overlap. Hollow markers: gudhi on the prefix T'.

S4.3 W3: cavities (E3)

mmhm has the lowest amortised latency at every size of W3, beyond noise, but the DRB is not uniformly better than recomputation. We take full-sorted as the reference here: it is faster than full-id at every size and drifts less on the larger instances (Section S1.4). Against it, the DRB methods without rebuilds are slower at $N = 14$ and have a higher p99 at $N \leq 20$; the rebuilding ones are faster at every size.

At $N = 14$ ($|K_0| = 74{,}957$), mmhm takes 1,230 µs amortised, full-sorted 1,790 and full-id 2,320 µs; drb-id (3,180 µs) and mmhm-norb (2,600 µs) are slower than full-sorted, and at $N = 20$ drb-id still is (speedup 0.794). The p99 of full-sorted, 3,550 µs, is below that of every DRB method at $N = 14$ except mmhm (5,110 µs, within noise). At $N = 27$ ($|K_0| = 525{,}043$) every DRB method is faster than both static baselines on average. mmhm is 5.99 times faster than full-sorted over the whole run (7.45 and 8.44 over the first and the last 200 updates) and 9.46 times faster than full-id: full-id is 0.957 times as fast as full-sorted over the first 200 updates and 0.564 times as fast over the last 200, as its order drifts (Table S7). Over the same first T' updates mmhm is 160 times faster than gudhi. At $N = 27$ the p99 of mmhm and drb-id-rb is below that of full-sorted, that of mmhm-norb within noise of it, and that of drb-id above it. At $N = 20$ and $N = 27$ the amortised latencies are ordered mmhm, drb-id-rb, mmhm-norb, drb-id, all beyond noise (at $N = 14$ the last two are within noise): rebuilding matters more than the order, and the Morse order helps both with and without rebuilds.

The basis grows over a W3 run (Figure S3(b); it also grows on W1, E5). At $N = 20$, without rebuilds, $\mathrm{nnz}(R)+\mathrm{nnz}(V)$ grows from 812,580 to 4,505,833 (drb-id) and to 3,734,680 (mmhm-norb), and the latency grows with it. The growth trigger rebuilds mmhm 6.0 times and drb-id-rb 10 times per run (means over the runs), and the runs end at 1,483,381 and 1,667,205.

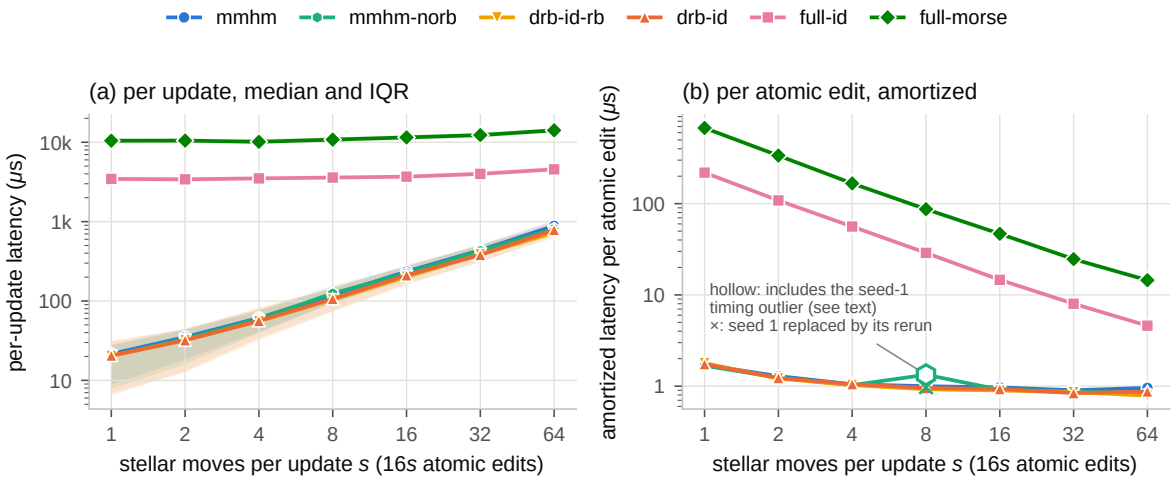


Figure S2: Cost against the number s of stellar moves per update (E2; W2, $N = 22$, $16s$ atomic edits per update; log–log). (a) Median per-update latency with interquartile range; (b) amortised latency per atomic edit. The hollow mmhm-norb point at $s = 8$ includes the outlier run of E1; $\times$ marks the same point with that run replaced by its rerun.

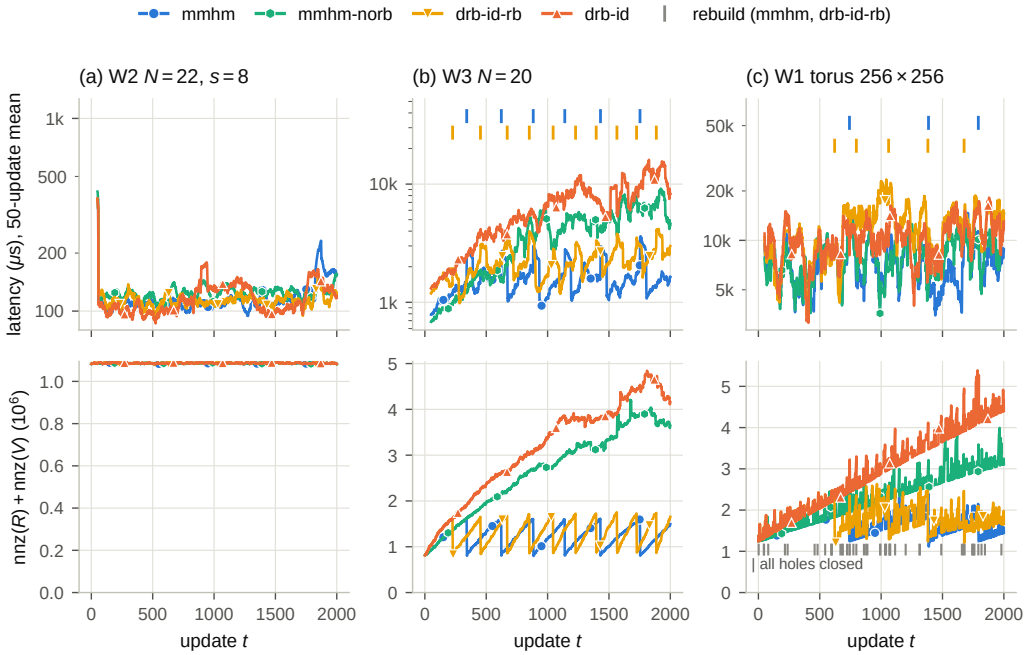


Figure S3: Behaviour over time of the four DRB methods (seed 0). Top: per-update latency, mean of a sliding window of 50 updates, with the rebuilds of mmhm and drb-id-rb marked. Bottom: $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ after each update. (a) W2 with $N = 22$, $s = 8$ (E1); (b) W3 with $N = 20$ (E3); (c) W1 torus 256×256 (E5), with the updates after which no hole is open marked.

S4.4 W4: flag complexes (E4)

The DRB methods are faster than both static baselines on every W4 instance, their latency hardly depends on $|K_0|$, and on the larger instances the Morse order helps. In the square with $D = 2$ (Figure S1(c)), their median latency grows with slope 0.02 to 0.15, against 0.97 for full-id (three sizes, so the slopes are indicative only). Against full-sorted, our reference here because it drifts less than full-id on every W4 instance (Table S7), their speedup is 3.25 to 6.09 at $n = 1000$ and 13.3 to 132 at $n \geq 4000$; against full-id it is 3.72 to 7.64 and 12.2 to 109. At $n = 16000$ full-sorted is slower than full-id (14,400 against 12,000 µs amortised), and mmhm is 114 times faster than it over the whole run and 81.2 and 120 times over the first and the last 200 updates (against full-id: 94.5, 64.8 and 103). The p99 of the DRB methods is below that of both baselines on every W4 instance except the two with $n = 1000$ and $D = 3$, where the difference is within noise (Table S12).

mmhm-norb is faster than drb-id beyond noise on the square instances with $n \geq 4000$; on the others the difference is within noise. At $n = 16000$ the amortised latencies are 110 and 169 µs, the p99 latencies 353 and 644 µs, and the p99 of the column additions per update 1,130 and 2,130 (E8). Under the default policy, rebuilds fired in only one W4 run of each rebuilding method (in the cube, $n = 1000$, $D = 3$). Without rebuilds, mmhm does the same additions as mmhm-norb (Table S11) but is slower at $n = 16000$ (127 against 110 µs, beyond noise). We have not profiled the difference. One difference between the two is the index of cells by vertex tuple that mmhm maintains for a possible Morse rebuild; in E7, mmhm with both triggers off, which never rebuilds but keeps the index, has a median of 98.7 µs against 87.7 µs for mmhm-norb.

S5 Surfaces (W1, E5)

On average the DRB is at best about twice as fast as recomputation, sometimes slower, and it has the worse tail; the combinatorial counters are far cheaper than both. On no W1 instance is a DRB method more than 2.06 times faster than full-id, and on several smaller instances mmhm-norb is within noise of it. Against full-sorted, drb-id is slower on every W1 instance (within noise on the 256×256 torus), mmhm-norb is slower on the four smallest, and no DRB method is more than 2.00 times faster. On the largest sphere and torus the insertion-order variants are slower than full-id and the Morse-ordered ones faster, all beyond noise. Every DRB variant has a higher p99 than full-id on every W1 instance.

On the sphere with $L = 7$ ($|K_0| = 983{,}042$), full-id takes 30,700 µs amortised, drb-id 37,100 and drb-id-rb 39,200 µs; mmhm-norb and mmhm are 1.98 and 1.85 times faster than full-id. On the torus with $n = m = 500$ ($|K_0| = 1{,}500{,}000$) the speedups are 0.848 (drb-id), 0.663 (drb-id-rb), 1.13 (mmhm) and 1.14 (mmhm-norb). The median tells a different story from the average: at $L = 7$ the median of mmhm is 4.82 times below that of full-id (6,050 against 29,100 µs), while its p99 is 151,000 against 52,200 µs; the average is set by a heavy tail (Tables S4 and S12). euler needs 10.5 to 30.1 µs per update, including the edits of the complex store, 1,040 times less than full-id on the largest sphere.

The median of the DRB methods grows with $|K_0|$ almost like that of full-id (slopes 0.80 to 1.01 on the sphere, against 1.02; Figure S1(d)); the chain sizes behind this were not logged. The closed surface, on which Proposition 4 forces $\Omega(n_2)$ work, reappears after only 38.0 of the 2,000 updates of a run on the largest sphere (mean over the runs), too rarely to set the median, and on that sphere the median of mmhm-norb is nearly the same over the first and the last 200 updates (6,970 and 7,650 µs). The cause of the growth is open. Between rebuilds the basis also becomes denser (Figure S3(c)): on the largest torus $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ grows over a run by a factor 2.46 for drb-id and 1.84 for mmhm-norb. This is Remark 5.1 in practice: on connected surfaces with known connectivity, the counters are correct and much cheaper, and the algebra is not needed.

S6 Non-locality witnesses (W5, E6)

Where Proposition 4 says it does. On both witness families, the cost of every DRB method per toggle grows linearly with the size of the toggled cycle. The DRB methods stay faster than full-id on average, but their p99 is higher on the cycle graph, and for the Morse-ordered variants also on the largest sphere; only euler (on the sphere) is nearly independent of the size.

The amortised latency per toggle grows with slope 1.06 to 1.10 on W5a (L from 10^3 to 10^6) and 1.10 to 1.16 on W5b (levels 3 to 8), against 1.05 and 1.05 for full-id (Figure 4). At $L = 10^6$, drb-id takes 14,200 µs, mmhm 20,100 µs and full-id 36,300 µs; on the largest sphere ($|K_0| = 3{,}932{,}162$) the DRB methods take 21,500 to 29,400 µs and full-id 178,000 µs. On W5a the p99 of every DRB method exceeds that of full-id at every L, beyond noise except for mmhm at $L = 1000$ (Table S12); at $L = 10^6$ it is 101,000 µs for mmhm against 48,100 µs. On W5b at level 8 the p99 of mmhm and mmhm-norb is 1.13 and 1.15 times that of full-id, that of the insertion-order variants 0.849 and 0.859 times. The Morse order does not help here: at $L = 10^6$, drb-id is faster than mmhm beyond noise, and the reference implementation likewise counts more elementary operations per toggle under the Morse order than under insertion order on these families (`reference/README.md`); we have not identified why. euler takes 0.106 to 0.267 µs per toggle on W5b (slope 0.14).

S7 Rebuild policy (E7)

It depends on the workload, and no setting is best everywhere. On W3, where the basis grows, rebuilding by the growth trigger pays off and the Morse order at a rebuild helps. On W2 and W4 the trigger hardly ever fires and periodic rebuilds only add cost (Figure S4). On the surfaces of W1, where the basis also grows, rebuilds help on the smaller instances and not on the largest (Table S8).

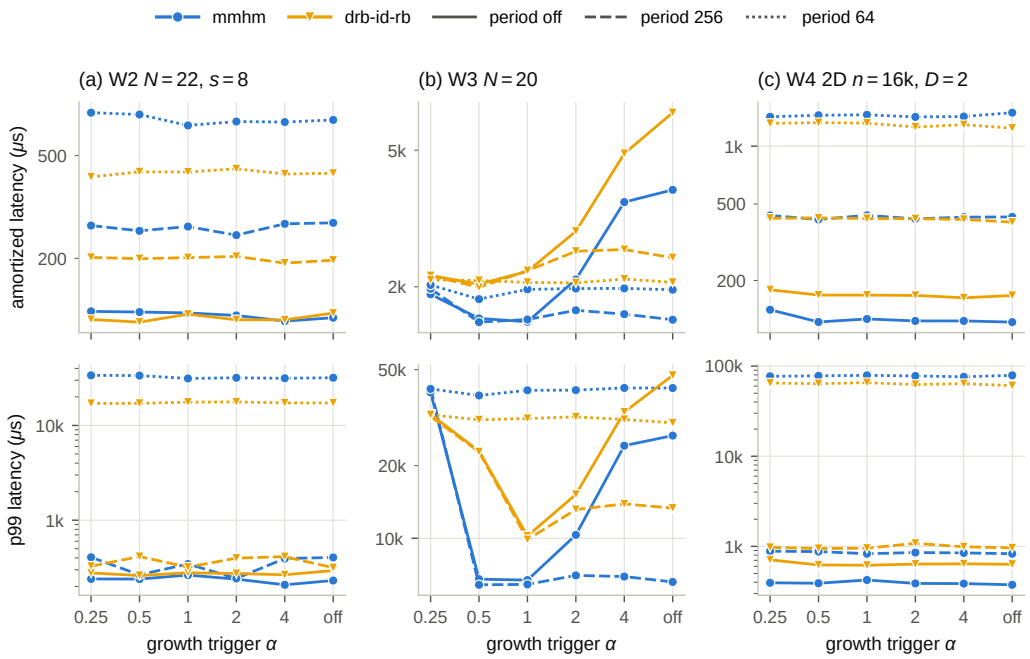


Figure S4: Rebuild policy (E7): amortised (top) and p99 (bottom) per-update latency of mmhm and drb-id-rb against the growth threshold α, one line per period m; three runs pooled. (a) W2 with $N = 22$, $s = 8$; (b) W3 with $N = 20$; (c) W4 in the square with $n = 16000$, $D = 2$. With $\alpha = m = \infty$ ("off") the methods never rebuild. E7 has no W1 instance; W1 rebuild data are in Table S8.

W2 and W4. With the growth trigger alone, no rebuild fired on W2 at any α, and on W4 one mmhm run of three rebuilt once, at $\alpha = 0.25$ (0.33 rebuilds per run). Periodic rebuilds raised the amortised latency of mmhm on W2 from 123 µs ($\alpha = 1$, no period) to 265 µs ($m = 256$) and 654 µs ($m = 64$). A Morse rebuild, which includes a collapse, costs more than an insertion-order one: drb-id-rb took 432 µs at $m = 64$. mmhm is faster than drb-id-rb at only 2 of the 18 settings on W2 and at 8 on W4.

W3. At $\alpha = 1$ without a period, mmhm took 1,580 µs amortised against 3,830 µs without rebuilds, and drb-id-rb 2,220 against 6,440 µs; the p99 of mmhm fell from 26,600 to 6,710 µs. Too small a threshold hurts again: at $\alpha = 0.25$ mmhm rebuilt 35 times per run and its p99 was 40,300 µs. On W3 mmhm is faster than drb-id-rb at 18 of the 18 settings. Under the default policy, the updates after the first rebuild need fewer additions than the same updates of the never-rebuilding twin (ratio 0.85 for mmhm, 0.78 for drb-id-rb at $N = 20$) and much less time (0.33 and 0.29), on a much sparser basis (Figure S3(b)).

W1. Under the default policy rebuilds fire on every W1 instance (Table S8). Against its never-rebuilding twin, mmhm is faster beyond noise on the sphere with $L \leq 5$ and the tori with $n \leq 128$ (for instance 837 against 1,910 µs at $L = 5$), within noise on the sphere with $L = 7$ and the tori with $n \geq 256$, and slower beyond noise on the sphere with $L = 6$. drb-id-rb is faster than drb-id on the sphere with $L \leq 5$ and the tori with $n \leq 128$, within noise on the spheres with $L \geq 6$, and slower beyond noise on the tori with $n \geq 256$. In the 100 updates after an mmhm rebuild, mmhm performs more additions than mmhm-norb on the same updates on every W1 instance except the sphere with $L = 5$, by factors up to 2.52. Over all updates after the first rebuild, the ratio of additions is below one on the large spheres (0.95 and 0.92) and above one on the largest torus (1.21), while the ratio of time is 1.10 and 1.09 on those spheres. The additions and the time disagree here, as the counters are only proxies (Section 5.1); we have not identified the mechanism.

S8 What the Morse order buys (E8)

At build time nothing against the insertion orders of our generators, although both structured orders are far better than a random one. Between rebuilds it depends on the workload: the Morse order makes the DRB faster on surfaces, on cavities and on the larger flag complexes, makes no difference on W2, and makes it slower on the non-locality witnesses.

At build time. Lemma 2 and Corollaries 2.1 and 2.2 describe the build, so we compare the builds of drb-id and mmhm-norb (Table S6). The insertion orders written by our generators already need almost no additions on W2 (192 at

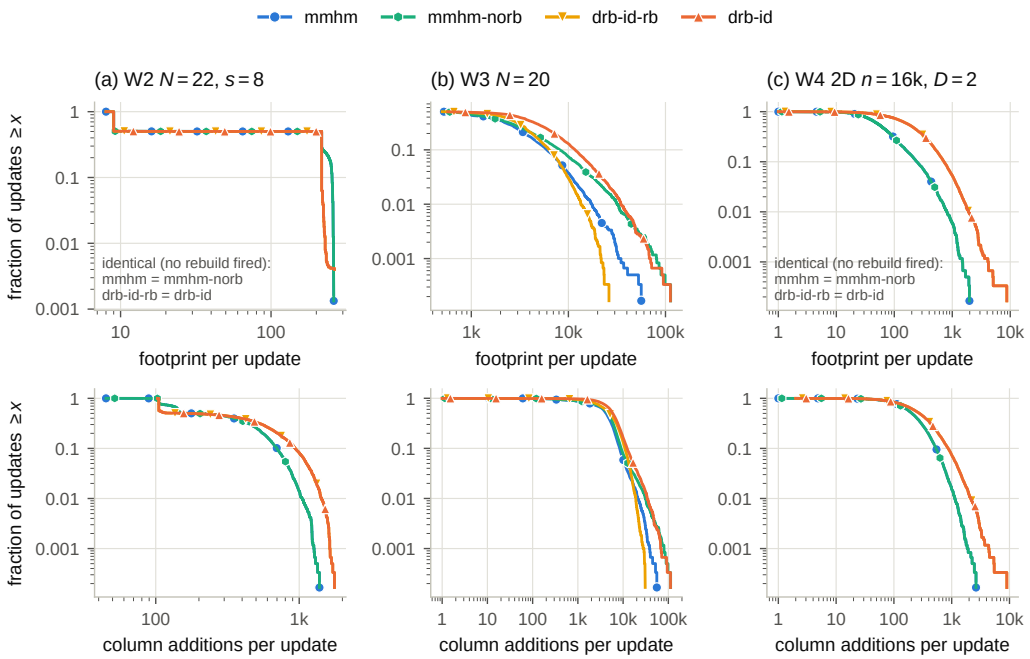


Figure S5: Counters between rebuilds (E8): complementary cumulative distributions of the footprint per update (top; the sum of $|\operatorname{occ}(\sigma)|$ over its deletions) and of the column additions per update (bottom), over the updates without a rebuild (three runs pooled; log–log). (a) W2 with $N = 22$, $s = 8$; (b) W3 with $N = 20$; (c) W4 in the square with $n = 16000$, $D = 2$. On W2 and W4 no rebuild fired, so mmhm coincides with mmhm-norb and drb-id-rb with drb-id. The W1 torus of Table S11 is not shown.

$N = 33$, against 0 in the Morse order) and none on W3; on W1 and W5 both orders need the same number within one percent (for instance 327,806 and 327,679 on the sphere with $L = 7$). On W4 the Morse order needs more additions, by factors 1.04 to 1.15, and yields a slightly smaller basis ($\operatorname{nnz}(R) + \operatorname{nnz}(V)$ ratio 0.963 to 0.995). The lemma and its corollaries thus say where the work of a build is done, not that there is less of it than in a good insertion order.

Against a random order. The insertion orders of our generators are themselves structured. Built in a uniformly random order (drb-rand, one instance per family, Table S14), the basis has 1.84 to 2.29 times as many entries after INIT as in either structured order, and the build needs more additions on every instance (on W2 72,361, against 192 in insertion and 0 in Morse order). On W1, W2 and W3 the updates are slower beyond noise, by factors 1.50 to 3.94 against either structured order (on W2 348 against 88.4 and 91.1 µs); there the choice between the two structured orders matters less than using one. On W4 drb-rand is within noise of drb-id and 1.48 times slower than mmhm-norb.

Between rebuilds. Comparing the two methods that never rebuild, mmhm-norb against drb-id, the Morse order is faster beyond noise on the W1 spheres with $L \geq 5$ and on all W1 tori (on the sphere with $L = 7$, 15,500 against 37,100 µs), on W3 with $N \geq 20$ and on W4 in the square with $n \geq 4000$. It is within noise on W2 (except on the instance with the outlier run of E1) and on the smaller W3 and W4 instances, and slower beyond noise on W5a for every $L \geq 3000$ (at $L = 10^6$, 20,300 against 14,200 µs) and on W5b from level 6 on. Figure S5 and Table S11 show the counters on the non-rebuild updates of one instance each of W2, W3, W4 and W1; the footprint of an update is the sum of $|\operatorname{occ}(\sigma)|$ over its deleted simplices. Without rebuilds the Morse order gives lighter tails of the column additions on all four: on W4 ($n = 16000$) the p99 falls from 2,130 to 1,130 and the median footprint from 204 to 60.0. Not every quantile improves: on W2 the median additions rise from 156 to 166 and the p99 footprint from 228 to 259, and with rebuilds the insertion-order rebuild of drb-id-rb leaves lighter footprint tails than the Morse rebuild (p99 13,400 against 17,300 on W3), although mmhm has the lower amortised latency there (E7). Between rebuilds the Morse order is a heuristic (Remark 4.3), and these measurements are its whole justification.

S9 Full result tables

Latencies are per update in µs, the three runs pooled; $'$ marks gudhi on the prefix $T' = 200$, and an empty cell a method not run in that experiment.

Table S6: The build of K_0 in insertion order (drb-id) and in the Morse order (mmhm-norb): $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ after INIT and column additions of the build, with the ratios Morse/insertion (E8).

			$\mathrm{nnz}(R) + \mathrm{nnz}(V)$ after INIT			additions of the INIT build		
exp.	instance	$\lvert K_0\rvert$	insertion	Morse	ratio	insertion	Morse	ratio
E1	W2 $N = 7$, $s = 8$	10,739	39,978	39,783	0.995	192	0	0.00
E1	W2 $N = 10$, $s = 8$	28,757	107,651	107,454	0.998	192	0	0.00
E1	W2 $N = 15$, $s = 8$	92,787	349,191	348,999	0.999	192	0	0.00
E1	W2 $N = 22$, $s = 8$	286,589	1,082,490	1,082,298	1.000	192	0	0.00
E1	W2 $N = 33$, $s = 8$	955,059	3,617,055	3,616,863	1.000	192	0	0.00
E2	W2 $N = 22$, $s = 1$	285,805	1,079,466	1,079,442	1.000	24	0	0.00
E2	W2 $N = 22$, $s = 2$	285,917	1,079,898	1,079,850	1.000	48	0	0.00
E2	W2 $N = 22$, $s = 4$	286,141	1,080,762	1,080,666	1.000	96	0	0.00
E2	W2 $N = 22$, $s = 16$	287,485	1,085,946	1,085,562	1.000	384	0	0.00
E2	W2 $N = 22$, $s = 32$	289,277	1,092,858	1,092,090	0.999	768	0	0.00
E2	W2 $N = 22$, $s = 64$	292,861	1,106,686	1,105,146	0.999	1,536	0	0.00
E3	W3 $N = 14$	74,957	281,778	281,778	1.000	0	0	–
E3	W3 $N = 20$	215,321	812,580	812,580	1.000	0	0	–
E3	W3 $N = 27$	525,043	1,986,039	1,986,039	1.000	0	0	–
E4	W4 2D $n = 1$k, $D = 2$	16,095	64,379	64,071	0.995	24,751	26,973	1.09
E4	W4 3D $n = 1$k, $D = 3$	22,831	101,754	98,034	0.963	20,025	21,680	1.08
E4	W4 2D $n = 1$k, $D = 3$	28,435	127,170	124,894	0.982	31,045	32,392	1.04
E4	W4 2D $n = 4$k, $D = 2$	63,935	262,582	257,571	0.981	99,634	107,925	1.08
E4	W4 3D $n = 4$k, $D = 3$	85,205	381,901	368,940	0.966	68,575	78,678	1.15
E4	W4 2D $n = 4$k, $D = 3$	111,960	508,866	495,893	0.975	123,174	128,327	1.04
E4	W4 2D $n = 16$k, $D = 2$	254,196	1,062,015	1,041,464	0.981	396,613	427,675	1.08
E5	W1 sphere $L = 4$	15,362	51,227	48,636	0.949	5,134	5,119	1.00
E5	W1 torus 64×64	24,576	78,131	77,984	0.998	8,317	8,359	1.01
E5	W1 sphere $L = 5$	61,442	203,499	194,556	0.956	20,510	20,479	1.00
E5	W1 torus 128×128	98,304	311,923	311,628	0.999	33,021	33,107	1.00
E5	W1 sphere $L = 6$	245,762	810,635	778,236	0.960	81,982	81,919	1.00
E5	W1 torus 256×256	393,216	1,246,451	1,245,856	1.000	131,581	131,751	1.00
E5	W1 sphere $L = 7$	983,042	3,235,275	3,112,956	0.962	327,806	327,679	1.00
E5	W1 torus 500×500	1,500,000	4,752,487	4,751,324	1.000	500,997	501,331	1.00
E6	W5a $L = 1{,}000$	2,000	3,997	3,997	1.000	999	999	1.00
E6	W5b level 3	3,842	12,915	12,156	0.941	1,286	1,279	0.99
E6	W5a $L = 3{,}000$	6,000	11,997	11,997	1.000	2,999	2,999	1.00
E6	W5b level 4	15,362	51,227	48,636	0.949	5,134	5,119	1.00
E6	W5a $L = 10{,}000$	20,000	39,997	39,997	1.000	9,999	9,999	1.00
E6	W5a $L = 30{,}000$	60,000	119,997	119,997	1.000	29,999	29,999	1.00
E6	W5b level 5	61,442	203,499	194,556	0.956	20,510	20,479	1.00
E6	W5a $L = 100{,}000$	200,000	399,997	399,997	1.000	99,999	99,999	1.00
E6	W5b level 6	245,762	810,635	778,236	0.960	81,982	81,919	1.00
E6	W5a $L = 300{,}000$	600,000	1,199,997	1,199,997	1.000	299,999	299,999	1.00
E6	W5b level 7	983,042	3,235,275	3,112,956	0.962	327,806	327,679	1.00
E6	W5a $L = 1{,}000{,}000$	2,000,000	3,999,997	3,999,997	1.000	999,999	999,999	1.00
E6	W5b level 8	3,932,162	12,926,027	12,451,836	0.963	1,310,974	1,310,719	1.00

Table S7: Drift (Section S1.4): median latency of full-id over the first and the last 200 updates and their ratio; the same ratio for full-sorted, full-morse, drb-id and mmhm; and the speedup of mmhm over full-id and over full-sorted on each of the two windows (ratio of their amortised latencies over the same updates).

		full-id med.		last/first block median					mmhm × full-id		mmhm × full-sorted	
exp.	instance	first	last	f-id	f-sorted	f-morse	drb-id	mmhm	first	last	first	last
E1	W2 $N = 7$, $s = 8$	136	134	0.99	1.04	0.81	1.01	0.90	2.44	2.03	2.66	2.35
E1	W2 $N = 10$, $s = 8$	373	376	1.01	1.03	0.99	0.98	0.94	5.01	5.62	5.48	6.55
E1	W2 $N = 15$, $s = 8$	1,170	1,240	1.06	1.01	0.96	1.08	1.08	10.9	12.9	12.0	13.1
E1	W2 $N = 22$, $s = 8$	3,470	3,760	1.08	0.98	0.97	0.99	1.12	19.5	28.3	23.0	28.0
E1	W2 $N = 33$, $s = 8$	14,800	14,800	1.00	1.01	1.00	0.99	1.04	46.0	103	44.2	100
E2	W2 $N = 22$, $s = 1$	3,360	3,540	1.05		0.96	0.93	0.90	42.5	158		
E2	W2 $N = 22$, $s = 2$	3,480	3,090	0.89		0.92	0.80	0.92	33.7	95.4		
E2	W2 $N = 22$, $s = 4$	3,560	3,450	0.97		0.98	0.87	0.96	29.3	58.0		
E2	W2 $N = 22$, $s = 16$	3,460	3,670	1.06		0.94	0.93	0.96	11.1	16.5		
E2	W2 $N = 22$, $s = 32$	3,780	3,940	1.04		1.00	1.14	1.01	7.80	8.93		
E2	W2 $N = 22$, $s = 64$	4,150	5,020	1.21		0.96	1.12	1.26	5.58	5.45		
E3	W3 $N = 14$	1,680	2,400	1.44	1.44	1.33	2.00	1.22	1.76	1.95	1.33	1.67
E3	W3 $N = 20$	3,830	8,490	2.22	1.84	1.65	4.31	1.41	3.79	5.59	3.25	4.04
E3	W3 $N = 27$	8,040	23,000	2.86	1.72	1.50	4.18	1.20	7.79	15.0	7.45	8.44
E4	W4 2D $n = 1$k, $D = 2$	536	877	1.64	1.04	1.00	1.29	1.77	7.99	7.71	8.59	5.23
E4	W4 3D $n = 1$k, $D = 3$	751	1,000	1.33	1.01	1.15	1.34	2.12	4.96	3.51	5.60	2.82
E4	W4 2D $n = 1$k, $D = 3$	932	1,220	1.31	1.17	1.18	1.33	2.24	6.20	3.18	6.49	3.29
E4	W4 2D $n = 4$k, $D = 2$	2,140	3,190	1.49	1.13	1.08	1.34	2.08	22.4	18.0	29.5	17.7
E4	W4 3D $n = 4$k, $D = 3$	2,630	3,530	1.34	1.03	1.28	1.15	1.61	16.2	14.7	21.0	14.1
E4	W4 2D $n = 4$k, $D = 3$	3,570	5,200	1.46	1.23	1.27	1.13	1.52	15.1	14.5	17.8	14.2
E4	W4 2D $n = 16$k, $D = 2$	10,800	12,300	1.15	1.12	1.21	1.10	1.14	64.8	103	81.2	120
E5	W1 sphere $L = 4$	362	425	1.18	1.09	1.04	1.99	0.84	1.90	2.28	1.32	1.54
E5	W1 torus 64×64	410	752	1.83	1.00	1.13	2.16	0.90	1.44	2.07	1.50	1.16
E5	W1 sphere $L = 5$	1,500	1,730	1.15	0.99	1.02	2.03	0.90	2.13	1.88	2.16	1.80
E5	W1 torus 128×128	1,930	2,350	1.21	1.08	1.15	1.88	0.89	1.51	1.73	1.40	1.47
E5	W1 sphere $L = 6$	6,280	6,540	1.04	1.05	1.10	1.16	0.98	2.30	1.88	2.30	1.85
E5	W1 torus 256×256	7,750	8,530	1.10	0.99	1.21	1.21	1.00	1.20	1.19	1.36	1.21
E5	W1 sphere $L = 7$	27,700	29,700	1.07	1.01	1.04	1.17	0.84	1.95	1.79	2.06	1.78
E5	W1 torus 500×500	32,600	33,600	1.03	1.03	1.05	1.08	1.18	1.17	0.919	1.26	0.966
E6	W5a $L = 1{,}000$	22.4	22.8	1.02			0.62	0.96	1.96	2.11		
E6	W5b level 3	108	112	1.03			0.89	0.97	11.0	12.1		
E6	W5a $L = 3{,}000$	67.3	67.3	1.00			1.16	0.82	1.89	1.92		
E6	W5b level 4	617	590	0.96			0.91	0.91	13.5	14.4		
E6	W5a $L = 10{,}000$	246	235	0.95			1.01	0.77	1.75	1.58		
E6	W5a $L = 30{,}000$	788	770	0.98			0.82	1.50	1.97	1.63		
E6	W5b level 5	2,620	2,730	1.04			0.72	0.98	11.9	12.1		
E6	W5a $L = 100{,}000$	2,610	2,640	1.01			0.64	1.05	1.64	1.84		
E6	W5b level 6	10,400	11,000	1.05			0.64	0.94	8.60	9.47		
E6	W5a $L = 300{,}000$	8,050	8,100	1.01			0.75	1.08	1.33	1.34		
E6	W5b level 7	46,400	46,000	0.99			0.55	0.62	6.91	7.37		
E6	W5a $L = 1{,}000{,}000$	36,400	36,200	0.99			0.94	1.33	1.78	1.78		
E6	W5b level 8	188,000	191,000	1.01			0.80	0.60	5.75	6.49		

Table S8: Does a rebuild help? Instances on which the default policy ($\alpha = 1$, no period) rebuilt: rebuilds per run (rb: drb-id-rb, m: mmhm); ratios of amortised latency to the never-rebuilding twin (rb/id: drb-id-rb over drb-id, m/norb: mmhm over mmhm-norb); the same ratio of column additions over the 100 updates after each rebuild; and the ratios of additions and of time over all updates after the first rebuild. Above 1: the rebuilding method was costlier.

	rebuilds / run		amortized latency		adds, 100 upd. after		all updates after the first rebuild			
							rb/id		m/norb	
instance	rb	m	rb/id	m/norb	rb/id	m/norb	adds	time	adds	time
W3 $N = 14$	24	13	0.46	0.47	0.98	1.00	0.99	0.40	1.01	0.41
W3 $N = 20$	10	6.0	0.34	0.41	0.72	0.82	0.78	0.29	0.85	0.33
W3 $N = 27$	4.3	3.0	0.54	0.62	0.71	0.79	0.76	0.46	0.79	0.47
W4 3D $n = 1$k, $D = 3$	0.33	0.33	1.16	1.00	0.94	0.54	1.01	0.95	0.84	0.70
W1 sphere $L = 4$	18	21	0.56	0.55	0.68	1.19	0.71	0.50	1.13	0.48
W1 sphere $L = 5$	16	10	0.62	0.44	0.72	0.91	0.71	0.56	0.83	0.38
W1 sphere $L = 6$	11	5.7	0.93	1.12	1.31	1.36	1.08	0.90	0.95	1.10
W1 sphere $L = 7$	9.0	2.3	1.06	1.07	2.44	2.04	1.79	1.05	0.92	1.09
W1 torus 64×64	16	15	0.49	0.56	0.52	1.07	0.54	0.43	1.01	0.50
W1 torus 128×128	7.7	6.7	0.83	0.68	0.50	1.11	0.48	0.80	0.91	0.61
W1 torus 256×256	4.3	3.0	1.13	0.93	0.70	1.50	0.54	1.17	1.03	0.86
W1 torus 500×500	1.7	1.0	1.28	1.02	0.85	2.52	0.69	1.44	1.21	1.02

Table S9: Median per-update latency (µs) of every instance of E1–E6.

exp.	instance	$\lvert K_0 \rvert$	full-id	full-sorted	full-morse	gudhi	drb-id	drb-id-rb	mmhm-norb	mmhm	euler
E1	W2 $N = 7$, $s = 8$	10,739	130	155	342	2,700	40.9	41.8	45.8	49.3	
E1	W2 $N = 10$, $s = 8$	28,757	373	398	907	7,970	50.9	54.5	60.1	65.3	
E1	W2 $N = 15$, $s = 8$	92,787	1,210	1,270	3,070	28,700	80.7	85.1	87.8	93.2	
E1	W2 $N = 22$, $s = 8$	286,589	3,590	3,730	10,800	94,100	107	104	125	119	
E1	W2 $N = 33$, $s = 8$	955,059	14,600	14,300	61,200	305,000′	132	131	143	142	
E2	W2 $N = 22$, $s = 1$	285,805	3,450		10,500		20.6	20.3	20.4	21.5	
E2	W2 $N = 22$, $s = 2$	285,917	3,410		10,500		32.0	32.0	33.5	35.0	
E2	W2 $N = 22$, $s = 4$	286,141	3,510		10,100		56.3	55.6	59.5	61.1	
E2	W2 $N = 22$, $s = 8$	286,589	3,590	3,730	10,800	94,100	107	104	125	119	
E2	W2 $N = 22$, $s = 16$	287,485	3,690		11,500		211	203	224	235	
E2	W2 $N = 22$, $s = 32$	289,277	4,000		12,300		381	387	429	437	
E2	W2 $N = 22$, $s = 64$	292,861	4,560		14,100		789	724	814	884	
E3	W3 $N = 14$	74,957	2,290	1,690	4,240	27,300	2,220	1,070	1,780	859	
E3	W3 $N = 20$	215,321	7,300	5,030	15,200	90,100	3,460	1,460	2,170	1,040	
E3	W3 $N = 27$	525,043	17,600	10,500	38,600	164,000′	2,690	1,620	1,790	1,170	
E4	W4 2D $n = 1$k, $D = 2$	16,095	807	614	1,050	6,160	88.6	86.2	82.5	89.7	
E4	W4 2D $n = 1$k, $D = 3$	28,435	1,170	1,070	1,790	12,800	171	168	157	162	
E4	W4 2D $n = 4$k, $D = 2$	63,935	2,820	2,820	4,720	34,200	109	110	89.7	93.8	
E4	W4 2D $n = 4$k, $D = 3$	111,960	4,570	4,780	8,810	71,100	211	205	169	183	
E4	W4 2D $n = 16$k, $D = 2$	254,196	11,700	13,700	24,300	194,000	131	130	87.7	102	
E4	W4 3D $n = 1$k, $D = 3$	22,831	1,020	858	1,550	10,500	120	124	121	126	
E4	W4 3D $n = 4$k, $D = 3$	85,205	3,100	3,220	6,150	50,400	154	161	132	139	
E5	W1 sphere $L = 4$	15,362	407	259	426	2,890	224	117	230	98.2	9.30
E5	W1 sphere $L = 5$	61,442	1,640	1,440	1,880	15,600	599	340	639	331	16.7
E5	W1 sphere $L = 6$	245,762	6,410	6,110	8,570	65,300	1,760	1,470	1,410	1,230	25.2
E5	W1 sphere $L = 7$	983,042	29,100	29,400	58,900	272,000′	7,340	7,520	7,160	6,050	30.5
E5	W1 torus 64×64	24,576	647	411	664	4,660	380	167	336	166	10.8
E5	W1 torus 128×128	98,304	2,090	1,830	3,330	23,800	716	585	684	533	18.4
E5	W1 torus 256×256	393,216	7,660	8,080	15,300	91,000′	2,170	2,540	2,200	2,090	26.8
E5	W1 torus 500×500	1,500,000	34,700	36,700	94,600	421,000′	10,900	13,000	11,700	11,500	32.1
E6	W5a $L = 1{,}000$	2,000	22.5			188	3.44	3.34	15.3	12.1	
E6	W5a $L = 3{,}000$	6,000	67.3			601	10.3	12.5	34.8	36.0	
E6	W5a $L = 10{,}000$	20,000	242			2,100	43.4	35.8	150	135	
E6	W5a $L = 30{,}000$	60,000	772			7,140	157	161	524	448	
E6	W5a $L = 100{,}000$	200,000	2,640			26,500	661	668	1,020	1,710	
E6	W5a $L = 300{,}000$	600,000	8,050			90,800′	1,990	2,070	4,170	4,310	
E6	W5a $L = 1{,}000{,}000$	2,000,000	35,900			372,000′	11,400	11,000	13,500	13,500	
E6	W5b level 3	3,842	110			737	4.38	5.54	5.80	4.51	0.0960
E6	W5b level 4	15,362	650			2,790	21.1	20.2	22.2	22.6	0.111
E6	W5b level 5	61,442	2,670			12,700	109	104	120	124	0.0930
E6	W5b level 6	245,762	10,800			60,900	564	555	743	735	0.0910
E6	W5b level 7	983,042	46,100			269,000′	2,570	2,460	3,810	3,450	0.105
E6	W5b level 8	3,932,162	188,000			1,460,000′	15,600	16,600	29,800	29,300	0.108

Table S10: Amortised per-update latency (µs) of every instance of E1–E6. The mmhm-norb entry for W2 with $N = 22$, $s = 8$ (in E1 and E2) includes the outlier run of Section S4.1. gudhi entries marked $'$ cover only the first T' updates and are not comparable with the others.

exp.	instance	$\lvert K_0\rvert$	full-id	full-sorted	full-morse	gudhi	drb-id	drb-id-rb	mmhm-norb	mmhm	euler
E1	W2 $N = 7$, $s = 8$	10,739	135	172	350	2,770	46.8	47.7	49.8	59.0	
E1	W2 $N = 10$, $s = 8$	28,757	387	413	944	8,190	56.5	59.7	65.3	71.6	
E1	W2 $N = 15$, $s = 8$	92,787	1,240	1,300	3,170	29,100	88.4	94.9	91.1	97.0	
E1	W2 $N = 22$, $s = 8$	286,589	3,700	3,960	11,100	97,600	122	118	171	128	
E1	W2 $N = 33$, $s = 8$	955,059	14,900	14,600	62,800	307,000′	165	163	163	165	
E2	W2 $N = 22$, $s = 1$	285,805	3,500		10,800		27.9	28.6	26.8	27.4	
E2	W2 $N = 22$, $s = 2$	285,917	3,470		10,800		39.3	38.8	39.7	41.2	
E2	W2 $N = 22$, $s = 4$	286,141	3,590		10,700		67.4	65.7	65.6	67.1	
E2	W2 $N = 22$, $s = 8$	286,589	3,700	3,960	11,100	97,600	122	118	171	128	
E2	W2 $N = 22$, $s = 16$	287,485	3,750		12,000		238	229	234	248	
E2	W2 $N = 22$, $s = 32$	289,277	4,100		12,600		430	440	450	461	
E2	W2 $N = 22$, $s = 64$	292,861	4,710		14,800		895	805	862	981	
E3	W3 $N = 14$	74,957	2,320	1,790	4,320	28,100	3,180	1,460	2,600	1,230	
E3	W3 $N = 20$	215,321	7,170	5,150	15,400	90,500	6,490	2,180	3,820	1,570	
E3	W3 $N = 27$	525,043	17,100	10,800	39,200	168,000′	4,600	2,500	2,920	1,810	
E4	W4 2D $n = 1$k, $D = 2$	16,095	822	655	1,100	6,280	115	116	107	122	
E4	W4 2D $n = 1$k, $D = 3$	28,435	1,190	1,120	1,810	13,000	296	301	275	289	
E4	W4 2D $n = 4$k, $D = 2$	63,935	2,830	2,960	4,850	34,700	140	146	113	126	
E4	W4 2D $n = 4$k, $D = 3$	111,960	4,610	4,990	8,980	73,200	376	370	277	299	
E4	W4 2D $n = 16$k, $D = 2$	254,196	12,000	14,400	25,100	196,000	169	168	110	127	
E4	W4 3D $n = 1$k, $D = 3$	22,831	1,060	924	1,590	11,000	244	284	241	241	
E4	W4 3D $n = 4$k, $D = 3$	85,205	3,160	3,440	6,290	51,300	236	245	192	201	
E5	W1 sphere $L = 4$	15,362	420	296	457	2,910	388	216	379	207	10.5
E5	W1 sphere $L = 5$	61,442	1,730	1,630	2,000	15,800	2,230	1,390	1,910	837	17.6
E5	W1 sphere $L = 6$	245,762	6,830	6,610	9,170	66,300	8,640	8,070	3,500	3,920	23.7
E5	W1 sphere $L = 7$	983,042	30,700	31,100	61,500	272,000′	37,100	39,200	15,500	16,600	29.5
E5	W1 torus 64×64	24,576	645	466	729	4,760	789	388	695	386	11.6
E5	W1 torus 128×128	98,304	2,260	2,060	3,570	24,400	2,950	2,450	2,420	1,650	18.5
E5	W1 torus 256×256	393,216	8,640	8,980	16,300	91,800′	10,300	11,600	7,680	7,170	26.2
E5	W1 torus 500×500	1,500,000	38,200	40,000	98,200	427,000′	45,100	57,600	33,400	34,000	30.1
E6	W5a $L = 1{,}000$	2,000	24.0			199	9.25	7.18	11.6	10.9	
E6	W5a $L = 3{,}000$	6,000	66.0			624	27.0	22.8	35.2	33.5	
E6	W5a $L = 10{,}000$	20,000	249			2,060	85.1	81.8	127	138	
E6	W5a $L = 30{,}000$	60,000	803			7,180	289	283	441	460	
E6	W5a $L = 100{,}000$	200,000	2,700			26,800	1,070	1,030	1,520	1,500	
E6	W5a $L = 300{,}000$	600,000	8,100			91,200′	3,500	3,590	5,910	5,950	
E6	W5a $L = 1{,}000{,}000$	2,000,000	36,300			376,000′	14,200	14,000	20,300	20,100	
E6	W5b level 3	3,842	118			762	9.72	11.1	10.6	9.67	0.106
E6	W5b level 4	15,362	604			2,860	40.9	42.0	40.6	41.6	0.127
E6	W5b level 5	61,442	2,520			12,600	229	232	205	214	0.142
E6	W5b level 6	245,762	10,200			61,500	951	951	1,120	1,100	0.173
E6	W5b level 7	983,042	43,800			270,000′	4,900	4,830	6,180	5,890	0.240
E6	W5b level 8	3,932,162	178,000			1,460,000′	21,500	21,700	29,400	28,700	0.267

Table S11: Counters between rebuilds (E8): median and p99 of the column additions and of the footprint ($\sum \lvert \mathrm{occ}(\sigma)\rvert$ over the deletions) per update, over the updates without a rebuild, and $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ after INIT and at the end (means over the runs).

instance	method	additions / update		footprint / update		$\mathrm{nnz}(R) + \mathrm{nnz}(V)$	
		median	p99	median	p99	after INIT	end
W2 $N = 22$, $s = 8$	drb-id	156	1,420	118	228	1,080,000	1,090,000
	drb-id-rb	156	1,420	118	228	1,080,000	1,090,000
	mmhm-norb	166	1,050	88.5	259	1,080,000	1,090,000
	mmhm	166	1,050	88.5	259	1,080,000	1,090,000
W3 $N = 20$	drb-id	5,910	34,000	585	34,700	813,000	4,510,000
	drb-id-rb	4,990	19,000	634	13,400	813,000	1,670,000
	mmhm-norb	4,570	30,900	574	31,600	813,000	3,730,000
	mmhm	4,200	21,100	574	17,300	813,000	1,480,000
W4 2D $n = 16$k, $D = 2$	drb-id	276	2,130	204	1,910	1,060,000	1,260,000
	drb-id-rb	276	2,130	204	1,910	1,060,000	1,260,000
	mmhm-norb	204	1,130	60.0	801	1,040,000	1,300,000
	mmhm	204	1,130	60.0	801	1,040,000	1,300,000
W1 torus 256×256	drb-id	877	8,290	55.5	1,910	1,250,000	4,480,000
	drb-id-rb	370	7,620	56.0	618	1,250,000	1,740,000
	mmhm-norb	639	4,360	55.0	1,570	1,250,000	3,300,000
	mmhm	402	4,770	58.0	754	1,250,000	1,490,000

Table S12: 99th-percentile per-update latency (µs) of every instance of E1–E6. The mmhm-norb entry for W2 with $N = 22$, $s = 8$ includes the outlier run of Section S4.1.

exp.	instance	$\lvert K_0\rvert$	full-id	full-sorted	full-morse	gudhi	drb-id	drb-id-rb	mmhm-norb	mmhm	euler
E1	W2 $N = 7$, $s = 8$	10,739	235	289	610	4,550	113	130	112	129	
E1	W2 $N = 10$, $s = 8$	28,757	617	742	1,760	13,500	136	132	147	153	
E1	W2 $N = 15$, $s = 8$	92,787	1,810	1,900	5,040	46,400	205	230	177	188	
E1	W2 $N = 22$, $s = 8$	286,589	5,730	6,730	19,000	184,000	299	269	1,230	253	
E1	W2 $N = 33$, $s = 8$	955,059	21,600	21,900	97,900	386,000′	404	357	314	333	
E2	W2 $N = 22$, $s = 1$	285,805	5,280		18,900		74.1	77.7	72.0	68.0	
E2	W2 $N = 22$, $s = 2$	285,917	5,400		18,100		97.7	96.7	92.9	93.1	
E2	W2 $N = 22$, $s = 4$	286,141	5,140		17,700		168	161	133	137	
E2	W2 $N = 22$, $s = 8$	286,589	5,730	6,730	19,000	184,000	299	269	1,230	253	
E2	W2 $N = 22$, $s = 16$	287,485	6,030		20,600		537	535	443	455	
E2	W2 $N = 22$, $s = 32$	289,277	6,280		21,600		971	985	900	842	
E2	W2 $N = 22$, $s = 64$	292,861	7,240		28,600		2,250	1,670	1,640	1,930	
E3	W3 $N = 14$	74,957	3,590	3,550	7,000	46,900	15,900	10,400	13,500	5,110	
E3	W3 $N = 20$	215,321	12,500	9,710	29,000	143,000	47,300	9,720	27,200	6,500	
E3	W3 $N = 27$	525,043	30,800	20,100	69,800	232,000′	29,600	11,700	18,600	7,400	
E4	W4 2D $n = 1$k, $D = 2$	16,095	1,480	1,270	1,810	10,000	491	500	461	527	
E4	W4 2D $n = 1$k, $D = 3$	28,435	2,020	1,970	2,950	20,800	1,900	2,160	1,790	1,840	
E4	W4 2D $n = 4$k, $D = 2$	63,935	4,690	5,030	7,510	50,400	566	604	426	543	
E4	W4 2D $n = 4$k, $D = 3$	111,960	7,920	8,490	14,400	141,000	2,440	2,460	1,740	1,900	
E4	W4 2D $n = 16$k, $D = 2$	254,196	19,400	24,300	42,200	262,000	644	652	353	402	
E4	W4 3D $n = 1$k, $D = 3$	22,831	1,750	1,700	2,320	18,800	2,000	2,160	1,920	1,820	
E4	W4 3D $n = 4$k, $D = 3$	85,205	4,920	6,190	10,400	76,400	1,490	1,460	1,020	1,020	
E5	W1 sphere $L = 4$	15,362	778	703	1,050	4,310	2,550	2,030	2,390	1,780	36.6
E5	W1 sphere $L = 5$	61,442	3,190	4,110	4,720	26,600	21,900	15,400	21,200	8,300	55.2
E5	W1 sphere $L = 6$	245,762	12,700	12,800	22,000	101,000	80,500	84,700	31,300	38,600	64.1
E5	W1 sphere $L = 7$	983,042	52,200	53,500	114,000	333,000′	305,000	355,000	128,000	151,000	73.9
E5	W1 torus 64×64	24,576	1,210	1,160	1,850	7,840	6,580	3,620	5,800	3,860	42.7
E5	W1 torus 128×128	98,304	4,830	4,880	8,600	38,500	26,800	20,900	25,800	19,400	59.4
E5	W1 torus 256×256	393,216	19,800	19,800	38,300	127,000′	78,800	89,400	92,000	88,400	68.2
E5	W1 torus 500×500	1,500,000	79,000	78,800	183,000	546,000′	313,000	420,000	401,000	397,000	74.5
E6	W5a $L = 1{,}000$	2,000	54.2			344	173	143	73.3	70.3	
E6	W5a $L = 3{,}000$	6,000	96.8			903	486	376	230	238	
E6	W5a $L = 10{,}000$	20,000	379			3,080	1,360	1,340	809	952	
E6	W5a $L = 30{,}000$	60,000	1,210			10,600	4,270	4,260	2,750	3,170	
E6	W5a $L = 100{,}000$	200,000	3,710			38,200	14,500	14,300	9,170	9,110	
E6	W5a $L = 300{,}000$	600,000	11,800			121,000′	44,600	45,500	29,200	29,500	
E6	W5a $L = 1{,}000{,}000$	2,000,000	48,100			457,000′	154,000	151,000	103,000	101,000	
E6	W5b level 3	3,842	216			1,190	120	139	172	188	0.202
E6	W5b level 4	15,362	983			4,560	463	469	656	665	0.747
E6	W5b level 5	61,442	3,760			18,800	2,960	2,980	2,930	3,020	1.40
E6	W5b level 6	245,762	15,700			87,800	9,910	11,800	13,800	13,700	2.24
E6	W5b level 7	983,042	64,900			338,000′	51,000	50,700	63,300	61,500	3.54
E6	W5b level 8	3,932,162	240,000			1,650,000′	203,000	206,000	276,000	270,000	4.07

Table S13: W4: ranges of the Betti numbers over all states of the three runs, and β_D/n at INIT. β_D is dominated by unfilled boundaries of $(D+2)$-cliques (Section S1.3).

instance	$\lvert K_0\rvert$	β_0	β_1	β_2	β_3	β_D/n at INIT
W4 2D $n = 1$k, $D = 2$	16,095	1–2	61–86	5,696–6,612	0	6.17
W4 2D $n = 4$k, $D = 2$	63,935	1–3	308–353	23,714–25,167	0	6.07
W4 2D $n = 16$k, $D = 2$	254,196	2–5	1,343–1,410	95,351–96,175	0	5.97
W4 2D $n = 1$k, $D = 3$	28,435	1–2	61–86	0	4,960–7,441	6.17
W4 2D $n = 4$k, $D = 3$	111,960	1–3	308–353	0–1	22,252–25,967	5.94
W4 3D $n = 1$k, $D = 3$	22,831	1–4	133–165	0–10	3,214–5,259	3.51
W4 3D $n = 4$k, $D = 3$	85,205	1–7	654–745	8–27	10,091–12,104	2.83

Table S14: Build order (E8): one instance per family built in insertion order (drb-id), in the Morse order (mmhm-norb) and in a random order (drb-rand): $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ after INIT, column additions of the build, and amortised latency (three runs pooled).

		$\mathrm{nnz}(R) + \mathrm{nnz}(V)$ after INIT			additions of the INIT build			amortized latency (µs)		
instance	$\lvert K_0\rvert$	insertion	Morse	random	insertion	Morse	random	insertion	Morse	random
W1 torus 128×128	98,304	311,923	311,628	624,407	33,021	33,107	48,963	2,950	2,420	8,740
W2 $N = 15$, $s = 8$	92,787	349,191	348,999	794,651	192	0	72,361	88.4	91.1	348
W3 $N = 14$	74,957	281,778	281,778	645,340	0	0	59,696	3,180	2,600	4,760
W4 2D $n = 4$k, $D = 2$	63,935	262,582	257,571	482,480	99,634	107,925	212,567	140	113	167

References

[1] Clément Maria, Jean-Daniel Boissonnat, Marc Glisse, and Mariette Yvinec. The Gudhi library: Simplicial complexes and persistent homology. In *Mathematical Software – ICMS 2014*, volume 8592 of *Lecture Notes in Computer Science*, pages 167–174. Springer, 2014.

Table S2: Repeated identical runs: coefficients of variation of the median, amortised and p99 latency.

instance	method	runs	CV median	CV amortized	CV p99
w1_torus_256x256_k2_h3	drb-id	5	1.6%	0.6%	2.9%
w1_torus_256x256_k2_h3	full-id	5	2.6%	2.3%	3.5%
w1_torus_256x256_k2_h3	mmhm	5	2.0%	2.5%	1.1%
w2_N22_s8	full-id	5	4.1%	4.5%	14.0%
w2_N22_s8	mmhm	5	7.1%	8.4%	12.0%
w2_N33_s8	full-id	3	2.3%	2.8%	4.3%
w2_N33_s8	mmhm	3	2.2%	3.0%	9.7%
w3_N20_k2_c4	drb-id	5	1.6%	1.7%	2.4%
w3_N20_k2_c4	full-id	5	3.2%	2.9%	4.8%
w3_N20_k2_c4	mmhm	5	5.9%	6.7%	8.2%
w5a_cycle_L100000	drb-id	5	3.8%	5.4%	1.9%
w5a_cycle_L100000	full-id	5	0.8%	1.3%	10.5%
w5a_cycle_L100000	mmhm	5	6.0%	4.2%	2.0%
w5a_L1000 (3 seeds = repetitions)	drb-id	3	25.6%	24.1%	20.2%
w5a_L1000 (3 seeds = repetitions)	drb-id-rb	3	1.5%	28.9%	33.5%
w5a_L1000 (3 seeds = repetitions)	full-id	3	14.4%	21.8%	39.6%
w5a_L1000 (3 seeds = repetitions)	gudhi	3	11.1%	4.0%	9.4%
w5a_L1000 (3 seeds = repetitions)	mmhm	3	31.9%	16.8%	22.9%
w5a_L1000 (3 seeds = repetitions)	mmhm-norb	3	21.9%	1.6%	3.2%
w5a_L10000 (3 seeds = repetitions)	drb-id	3	0.5%	4.3%	8.0%
w5a_L10000 (3 seeds = repetitions)	drb-id-rb	3	11.9%	2.5%	0.8%
w5a_L10000 (3 seeds = repetitions)	full-id	3	4.3%	6.8%	5.3%
w5a_L10000 (3 seeds = repetitions)	gudhi	3	11.9%	8.9%	6.0%
w5a_L10000 (3 seeds = repetitions)	mmhm	3	16.0%	7.7%	13.0%
w5a_L10000 (3 seeds = repetitions)	mmhm-norb	3	24.2%	6.5%	17.0%
w5a_L100000 (3 seeds = repetitions)	drb-id	3	2.8%	4.7%	8.0%
w5a_L100000 (3 seeds = repetitions)	drb-id-rb	3	2.6%	4.1%	2.3%
w5a_L100000 (3 seeds = repetitions)	full-id	3	1.1%	2.5%	5.3%
w5a_L100000 (3 seeds = repetitions)	gudhi	3	1.5%	2.3%	3.4%
w5a_L100000 (3 seeds = repetitions)	mmhm	3	7.5%	5.6%	4.4%
w5a_L100000 (3 seeds = repetitions)	mmhm-norb	3	4.4%	4.6%	11.3%
w5a_L1000000 (3 seeds = repetitions)	drb-id	3	15.6%	1.1%	0.6%
w5a_L1000000 (3 seeds = repetitions)	drb-id-rb	3	4.8%	1.1%	0.4%
w5a_L1000000 (3 seeds = repetitions)	full-id	3	2.1%	1.7%	2.4%
w5a_L1000000 (3 seeds = repetitions)	gudhi	3	2.1%	1.9%	4.7%
w5a_L1000000 (3 seeds = repetitions)	mmhm	3	14.7%	0.6%	0.2%
w5a_L1000000 (3 seeds = repetitions)	mmhm-norb	3	5.5%	1.1%	1.8%
w5a_L3000 (3 seeds = repetitions)	drb-id	3	13.3%	15.0%	15.2%
w5a_L3000 (3 seeds = repetitions)	drb-id-rb	3	1.9%	2.8%	2.2%
w5a_L3000 (3 seeds = repetitions)	full-id	3	13.4%	14.4%	19.4%
w5a_L3000 (3 seeds = repetitions)	gudhi	3	1.4%	3.4%	17.4%
w5a_L3000 (3 seeds = repetitions)	mmhm	3	18.9%	18.9%	19.3%
w5a_L3000 (3 seeds = repetitions)	mmhm-norb	3	14.4%	1.0%	1.4%
w5a_L30000 (3 seeds = repetitions)	drb-id	3	4.7%	4.6%	12.0%
w5a_L30000 (3 seeds = repetitions)	drb-id-rb	3	4.8%	7.5%	3.1%
w5a_L30000 (3 seeds = repetitions)	full-id	3	3.4%	5.2%	7.4%
w5a_L30000 (3 seeds = repetitions)	gudhi	3	2.3%	3.7%	1.6%
w5a_L30000 (3 seeds = repetitions)	mmhm	3	15.4%	8.1%	15.5%
w5a_L30000 (3 seeds = repetitions)	mmhm-norb	3	24.2%	1.5%	4.2%
w5a_L300000 (3 seeds = repetitions)	drb-id	3	6.6%	3.0%	0.6%
w5a_L300000 (3 seeds = repetitions)	drb-id-rb	3	2.3%	1.4%	0.5%
w5a_L300000 (3 seeds = repetitions)	full-id	3	1.1%	0.9%	3.1%
w5a_L300000 (3 seeds = repetitions)	gudhi	3	1.6%	1.3%	3.5%
w5a_L300000 (3 seeds = repetitions)	mmhm	3	2.4%	0.7%	1.3%
w5a_L300000 (3 seeds = repetitions)	mmhm-norb	3	4.6%	2.1%	2.0%

Table S3: Correctness (E0): per exact method, the runs, the Betti vectors they reported, the states compared with the full-id output of the same instance, with the closed-form values and with GUDHI, and the mismatches (drb-rand ran on four instances only).

method	runs	states	vs. full-id	vs. analytic	vs. GUDHI	mismatches
full-id	126	252,126	(ref.)	210,105	203,526	0
full-sorted	69	138,069	138,069	96,048	111,069	0
full-morse	87	174,087	174,087	132,066	147,087	0
gudhi	126	203,526	203,526	161,505	(ref.)	0
drb-id	126	252,126	252,126	210,105	203,526	0
drb-id-rb	288	576,288	576,288	426,213	527,688	0
drb-rand	12	24,012	24,012	18,009	24,012	0
mmhm-norb	126	252,126	252,126	210,105	203,526	0
mmhm	288	576,288	576,288	426,213	527,688	0
euler	42	84,042	84,042	84,042	57,042	0
all exact	1,290	2,532,690	2,280,564	1,974,411	2,005,164	0

Table S4: Summary at the largest size of each family of E1, E3, E4 and E5: $|K_0|$, the number T of timed updates per run, the median, p95 and amortised per-update latency in µs (three runs pooled), and the speedup over full-id (ratio of amortised latencies over all T updates). gudhi entries with $T = 200$ cover only the first T' updates, and on W3 full-id drifts (Table S7); the text gives matched comparisons. Every instance is listed in Tables S10 and S12.

instance		$\lvert K_0\rvert$	method	T	median	p95	amortized	× full-id
W1 sphere $L = 7$	(E5)	983,042	full-id	2,000	29,100	44,700	30,700	–
			full-sorted	2,000	29,400	44,800	31,100	0.986
			full-morse	2,000	58,900	82,000	61,500	0.499
			gudhi	200	272,000	317,000	272,000	0.113
			drb-id	2,000	7,340	218,000	37,100	0.827
			drb-id-rb	2,000	7,520	231,000	39,200	0.782
			mmhm-norb	2,000	7,160	82,000	15,500	1.98
			mmhm	2,000	6,050	86,100	16,600	1.85
			euler	2,000	30.5	49.7	29.5	1,040
W1 torus 500×500	(E5)	1,500,000	full-id	2,000	34,700	65,800	38,200	–
			full-sorted	2,000	36,700	66,600	40,000	0.955
			full-morse	2,000	94,600	125,000	98,200	0.389
			gudhi	200	421,000	503,000	427,000	0.0896
			drb-id	2,000	10,900	246,000	45,100	0.848
			drb-id-rb	2,000	13,000	335,000	57,600	0.663
			mmhm-norb	2,000	11,700	184,000	33,400	1.14
			mmhm	2,000	11,500	183,000	34,000	1.13
			euler	2,000	32.1	50.9	30.1	1,270
W2 $N = 33$, $s = 8$	(E1)	955,059	full-id	2,000	14,600	18,300	14,900	–
			full-sorted	2,000	14,300	19,000	14,600	1.02
			full-morse	2,000	61,200	80,400	62,800	0.237
			gudhi	200	305,000	354,000	307,000	0.0485
			drb-id	2,000	132	293	165	90.3
			drb-id-rb	2,000	131	285	163	91.5
			mmhm-norb	2,000	143	244	163	91.4
			mmhm	2,000	142	243	165	90.2
W3 $N = 27$	(E3)	525,043	full-id	2,000	17,600	25,500	17,100	–
			full-sorted	2,000	10,500	15,200	10,800	1.58
			full-morse	2,000	38,600	51,400	39,200	0.437
			gudhi	200	164,000	207,000	168,000	0.102
			drb-id	2,000	2,690	14,100	4,600	3.72
			drb-id-rb	2,000	1,620	6,460	2,500	6.85
			mmhm-norb	2,000	1,790	8,760	2,920	5.86
			mmhm	2,000	1,170	4,160	1,810	9.46
W4 2D $n = 16$k, $D = 2$	(E4)	254,196	full-id	2,000	11,700	15,300	12,000	–
			full-sorted	2,000	13,700	20,600	14,400	0.830
			full-morse	2,000	24,300	33,400	25,100	0.477
			gudhi	2,000	194,000	232,000	196,000	0.0611
			drb-id	2,000	131	393	169	71.0
			drb-id-rb	2,000	130	391	168	71.1
			mmhm-norb	2,000	87.7	236	110	109
			mmhm	2,000	102	265	127	94.5
W4 3D $n = 4$k, $D = 3$	(E4)	85,205	full-id	2,000	3,100	4,130	3,160	–
			full-sorted	2,000	3,220	4,880	3,440	0.921
			full-morse	2,000	6,150	8,150	6,290	0.503
			gudhi	2,000	50,400	62,500	51,300	0.0617
			drb-id	2,000	154	700	236	13.4
			drb-id-rb	2,000	161	740	245	12.9
			mmhm-norb	2,000	132	534	192	16.5
			mmhm	2,000	139	565	201	15.7

Table S5: The full E7 grid: amortised and p99 latency and rebuilds per run of mmhm and drb-id-rb for every growth threshold α and period m.

		period off			period 256			period 64		
method	α	rb	amort.	p99	rb	amort.	p99	rb	amort.	p99
W2 $N = 22$, $s = 8$ (latency in µs)										
mmhm	0.25	0	125	240	7.0	268	406	31	733	33,800
	0.5	0	124	239	7.0	256	265	31	721	33,600
	1	0	123	263	7.0	265	345	31	654	31,300
	2	0	120	239	7.0	246	243	31	678	31,900
	4	0	114	208	7.0	272	394	31	674	31,500
	off	0	118	231	7.0	274	404	31	687	31,900
drb-id-rb	0.25	0	116	277	7.0	202	327	31	413	17,100
	0.5	0	113	261	7.0	199	414	31	433	17,100
	1	0	121	278	7.0	201	320	31	432	17,600
	2	0	116	274	7.0	203	398	31	444	17,700
	4	0	115	266	7.0	192	413	31	424	17,300
	off	0	123	294	7.0	197	315	31	427	17,300
W3 $N = 20$ (latency in µs)										
mmhm	0.25	35	1,900	40,300	35	1,970	41,800	36	2,020	41,600
	0.5	15	1,620	6,770	15	1,580	6,410	31	1,840	39,100
	1	6.0	1,580	6,710	7.0	1,610	6,440	31	1,970	41,100
	2	2.3	2,100	10,300	7.0	1,710	7,020	31	1,980	41,100
	4	0.67	3,530	24,200	7.0	1,660	6,940	31	1,980	42,000
	off	0	3,830	26,600	7.0	1,600	6,590	31	1,960	42,000
drb-id-rb	0.25	45	2,160	32,700	45	2,160	32,000	45	2,090	32,500
	0.5	22	2,040	23,000	22	1,990	22,800	31	2,090	31,000
	1	10	2,220	10,200	10	2,230	9,880	31	2,060	31,400
	2	4.7	2,900	15,200	7.0	2,540	13,200	31	2,050	31,900
	4	1.0	4,890	33,400	7.0	2,570	13,900	31	2,110	31,100
	off	0	6,440	47,400	7.0	2,430	13,300	31	2,060	30,100
W4 2D $n = 16k$, $D = 2$ (latency in µs)										
mmhm	0.25	0.33	141	394	7.0	436	884	31	1,420	77,100
	0.5	0	122	390	7.0	416	875	31	1,450	78,000
	1	0	126	423	7.0	436	829	31	1,460	78,900
	2	0	123	388	7.0	418	854	31	1,420	77,800
	4	0	123	386	7.0	427	846	31	1,430	76,100
	off	0	121	375	7.0	428	829	31	1,490	79,000
drb-id-rb	0.25	0	179	709	7.0	421	974	31	1,310	64,900
	0.5	0	168	622	7.0	423	947	31	1,330	63,800
	1	0	168	618	7.0	420	953	31	1,320	65,500
	2	0	167	636	7.0	419	1,080	31	1,260	62,500
	4	0	162	640	7.0	416	988	31	1,290	63,900
	off	0	167	633	7.0	402	962	31	1,240	60,700